LEARN BEING FORWARD

Programming — TypeScript Programming

Interfaces & type aliases

COMPLETE LEARNING GUIDE

What You'll Learn

Interfaces and type aliases are TypeScript's two tools for giving a name to a type so it can be reused instead of repeated inline, and while they overlap significantly in capability, each has specific strengths that matter once a codebase grows. This sub-module covers defining object shapes with interfaces, the type alias syntax and the specific cases (unions, tuples, mapped/conditional types) where only a type alias will do, optional and readonly properties, the difference between extending an interface and intersecting type aliases, index signatures for dictionary-like objects, typing functions and callbacks, interface declaration merging (a capability unique to interfaces), and discriminated unions — one of the most practically useful patterns in the entire type system for modeling 'one of several distinct shapes' data safely.

1. Defining Object Shapes With Interfaces

An interface declares the shape a value must have — which property names exist, and what type each one must be — and is the idiomatic way to describe the contract for objects, function parameters, and class implementations. Any object (or class, via implements) satisfying that shape is considered compatible, regardless of how it was created, consistent with TypeScript's overall structural typing model.

TYPESCRIPT

```
interface User {
  id: number;
  name: string;
  email: string;
}

function greetUser(user: User): string {
  return `Hello, ${user.name}!`;
}

const u: User = { id: 1, name: "Mia", email: "mia@example.com" };
console.log(greetUser(u));
```

2. Type Aliases: A More General Naming Tool

A type alias (type Name = ...) can name any type — an object shape (functionally similar to an interface for this case), a union, a tuple, a primitive, or a complex mapped/conditional type. Where an interface is restricted to describing object-like and function-like shapes, a type alias is the only option for naming a union (type ID = string | number) or a tuple type, which is the most common reason to reach for `type` over `interface` in practice.

```
TYPESCRIPT
type ID = string | number;
type Point = [number, number];
type Callback = (error: Error | null, data?: string) => void;

type UserShape = {
  id: number;
  name: string;
};
```

3. Optional and Readonly Properties

A property marked with ? (email?: string) is optional — it may be omitted entirely from a conforming object, and its type inside the interface becomes string | undefined when accessed. readonly prevents a property from being reassigned after the object is created (readonly id: number), which the compiler enforces at the type level — it's a compile-time-only guarantee and has no runtime enforcement, so it doesn't prevent mutation through a type-cast or non-TypeScript code path, but it does catch accidental reassignment in ordinary TypeScript code.

- An optional property's accessed type is implicitly unioned with undefined — always check for its presence before using it directly.
- readonly is a compile-time-only restriction; it does not freeze the object at runtime the way Object.freeze() does in plain JavaScript.

```
TYPESCRIPT
interface Config {
  readonly apiUrl: string;
  timeout?: number;
}

const config: Config = { apiUrl: "https://api.example.com" };
// config.apiUrl = "other";   // error: cannot assign to 'apiUrl' because it is readonly
console.log(config.timeout ?? 5000); // handle the optional property's possible absence
```

4. Extending Interfaces vs. Intersecting Type Aliases

An interface extends another with the extends keyword, inheriting all of its members and allowing additional ones — this reads clearly as an 'is-a, plus more' relationship and produces clean, easy-to-read error messages when something doesn't match. A type alias achieves a similar combination using an intersection (&): type Admin = User & { permissions: string[] }. Both approaches generally produce equivalent results for simple object shapes, but interface extends tends to give cleaner compiler error messages for deeply nested hierarchies, which is part of why many style guides prefer `interface` for object shapes and reserve `type` for unions and other non-object types.

```typescript
interface User {
  id: number;
  name: string;
}

interface Admin extends User {
  permissions: string[];
}

const admin: Admin = { id: 1, name: "Dev", permissions: ["read", "write"] };

// Equivalent using a type alias and intersection:
type AdminAlt = User & { permissions: string[] };
```

5. Index Signatures for Dictionary-Like Objects

When an object's exact keys aren't known in advance — a lookup table keyed by arbitrary strings, for instance — an index signature ([key: string]: ValueType) describes that every property access with a string key returns a value of that type. This is distinct from Record<string, ValueType> (a built-in utility type covered in the Generics sub-module) only in syntax; both express the same 'arbitrary string key, fixed value type' shape, and either is a reasonable style choice.

```typescript
interface WordCounts {
  [word: string]: number;
}

const counts: WordCounts = {};
counts["typescript"] = 1;
counts["typescript"] += 1;
console.log(counts["typescript"]); // 2
console.log(counts["missing"]);     // undefined at runtime, though typed as `number`
```

6. Typing Functions as Properties

A function-typed property or standalone function type is written as (params) => ReturnType, both inside an interface/type alias and as a standalone named type. This is essential for typing callbacks, event handlers, and any object that holds behavior alongside data — getting the parameter types and return type right here is what allows the compiler to catch a mismatched callback signature passed into an API, rather than only discovering it at runtime.

```typescript
interface ApiClient {
  get(url: string): Promise<unknown>;
  onError?: (error: Error) => void;
}

type Comparator<T> = (a: T, b: T) => number;

const byLength: Comparator<string> = (a, b) => a.length - b.length;
["ccc", "a", "bb"].sort(byLength); // ["a", "bb", "ccc"]
```

7. Declaration Merging (Interfaces Only)

Declaring the same interface name more than once in the same scope merges all declarations into a single interface with every member combined — a capability unique to interfaces that type aliases deliberately do not support (redeclaring a type alias with the same name is a compile error). This is primarily useful for augmenting types from external libraries (adding a custom property to Express's Request type, for example) rather than something to rely on heavily in application code, where it can make a type's full shape harder to find by searching for a single declaration.

```typescript
interface Window {
  myAppVersion: string;
}

interface Window {
  myAppFeatureFlags: string[];
}

// Merged result: Window now has BOTH myAppVersion and myAppFeatureFlags
declare const win: Window;
console.log(win.myAppVersion, win.myAppFeatureFlags);
```

8. Discriminated Unions

A discriminated union combines several object-shape types that share one common 'discriminant' property with a distinct literal type in each member (a kind or type field) — the compiler uses that single property to automatically narrow which full shape applies inside a conditional or switch branch, giving full, safe access to each shape's unique fields with no manual type assertions. This pattern is one of the most valuable in the whole type system for modeling state that can legitimately be 'one of several different shapes' (a loading/success/error UI state, a set of distinct event types) without resorting to a pile of optional fields and

runtime undefined checks.

```typescript
type RequestState =
  | { status: "loading" }
  | { status: "success"; data: string[] }
  | { status: "error"; message: string };

function render(state: RequestState): string {
  switch (state.status) {
    case "loading":
      return "Loading...";
    case "success":
      return `Got ${state.data.length} items`;  // `data` is safely accessible here
    case "error":
      return `Error: ${state.message}`;          // `message` is safely accessible here
  }
}
```

Common Pitfalls

- Trying to use `interface` to name a union type — unions require a type alias; interfaces can only describe object/function shapes.
- Forgetting that an optional property's accessed type includes `undefined`, and using it without a presence check or default value.
- Assuming `readonly` provides runtime immutability like Object.freeze() — it is a compile-time-only check with zero runtime enforcement.
- Declaring the same interface twice unintentionally (e.g., a typo'd duplicate) and being confused why TypeScript merges them instead of erroring, unlike a duplicate type alias.
- Modeling multi-shape state with a pile of optional fields instead of a discriminated union, losing the compiler's ability to narrow and check exhaustiveness.
- Mixing up index signatures with Record<K, V> inconsistently across a codebase when either would do — pick one convention and stay consistent.

Key Takeaways

- Use `interface` for object/function shapes you might extend; use `type` for unions, tuples, and anything an interface structurally can't express.
- Optional (?) properties may be absent and are typed with an implicit `| undefined`; readonly properties can't be reassigned, but only at compile time.
- interface extends and type-alias intersection (&) both combine shapes — extends tends to give cleaner error messages for deep hierarchies.
- Index signatures ([key: string]: T) describe dictionary-like objects with arbitrary string keys and a consistent value type.

- Declaration merging is unique to interfaces and is mainly useful for augmenting third-party library types, not everyday application modeling.
- Discriminated unions with a shared literal 'kind' field let the compiler safely and exhaustively narrow between several distinct object shapes.

Learn More

TypeScript Handbook — Object Types
https://www.typescriptlang.org/docs/handbook/2/objects.html

TypeScript Handbook — Unions and Intersection Types
https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#union-types

TypeScript Handbook — Narrowing: Discriminated Unions
https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions